NP-hardness of total multi-output general Boolean circuit minimization under randomized many-one reductions

Rahul Ilango, Bruno Loff, Igor C. Oliveira

doi:10.4230/LIPIcs.CCC.2020.22; CC BY3.0

Editorial note. Rahul Ilango,Bruno Loff and IgorC.Oliveira. CCC2020 article22,DOI10.4230/LIPIcs.CCC.2020.22. CC BY3.0.

Editorial note. Faithful editable transcription of the official CCC2020 CC BY3.0 publication, article22 pp22:1–22:36. One total multi-output general-Boolean-circuit NP-hardness result, original Theorem4 under randomized many-one polynomial-time reductions. Full original circuit/generator/MultiMCSP/SetCover definitions and Theorem8 premise retained; complete Section3.2 arbitrary-generator warm-up with Lemmas9–11/Claim12 retained as grouped supporting context, not counted separately. Full Section4.1 definitions, full Section4.2 randomized reduction, polynomial-time construction analysis, decision-problem Footnote13, one-sided approximation guarantee, Proposition14 complete proof, Lemma15 proof and Lemma16 full short-description/counting proof retained. Truth-table/windows/lifts/canonical-DNF/evaluation-function conventions and all probability/parameter conditions preserved. All82 original bibliography entries retained. Original minor printed wording and numeric relaxations retained without mathematical repair, including Lemma16 description J-size bound versus later+2, n+2ell circuit inputs, and cover=4ell wording. Gate-type placeholder and concatenation typography adapted without changing operations. No statement that the reduction is deterministic or the target is single-output. Separate communication/learning/search-to-decision extensions not claimed by this item. Native not found in bounded lookup; transcription uses only licensed formal PDF. No required figures. Root handles compilation/database/publication.

Allowed standard prerequisites (editorial).

- Constant-factor hardness of bounded Set Cover. Original Theorem8[76] exact language/threshold promise retained.

Theorem 4 (Author Theorem4)**.** *Multi-MCSP is NP-hard under many-one randomized polynomial time reductions.*

Author circuit conventions and problems

A Boolean circuit consists of fan-in two AND gates, fan-in two OR gates, and NOT gates. The input gates are labelled by variables $x_1, \ldots, x_n$. We measure

the size of a Boolean circuit C, denoted $|C|$, using the number of AND, OR, and NOT gates in the circuit. (While this is the convention adopted here, our techniques are robust to modifications of the circuit size measure and of the gate types in the circuit.) The components of a multi-output Boolean function $f : \{0,1\}^n \to \{0,1\}^m$ are the single-output functions that compute the ith output bit of f for $i \in [m]$. We say a Boolean circuit C computes a multi-output Boolean function f if for each component f_i of f, there is a gate or input wire in C that computes f_i.

Definition 1 (Author Definition1)**.** *Multi-MCSP is defined as follows:*

Input. *Positive integers n, m, and s represented in unary, and a (multi-output) Boolean function $f : \{0,1\}^n \to \{0,1\}^m$ represented by a string of length $m \cdot 2^n$.*

Output. *The input is accepted if and only if there exists a Boolean circuit C of size at most s that computes f.*

Note that Multi-MCSP is in NP,[1] and that this problem is at least as hard as MCSP.

Boolean functions over an arbitrary set of generators We also consider circuit minimization for (single-output, total) Boolean functions $f : \{0,1\}^m \to \{0,1\}$ under an arbitrary set of generators. To explain, let V be a finite set, called the ground set, and $\mathcal{B} = \{B_i\}_{i\in[m]}$ a family of nonempty sets $B_i \subseteq V$ called generators, and let $A \subseteq V$. Then we let $D(A \mid \mathcal{B})$ denote the minimum number of unions, intersections, and complements that are required to construct A from the sets in $\mathcal{B}$. More precisely, the complement of a set $U \subseteq V$ is defined as $V \setminus U$, and we represent a construction of A from $\mathcal{B}$ as a sequence $B_1, \ldots, B_m, E_1, \ldots, E_\ell$ of sets in V such that $E_\ell = A$ and each set E_j is either the union or intersection of two previously generated sets, or the complement of a previously generated set. We assume for convenience that $D(B \mid \mathcal{B}) = 0$ if $B \in \mathcal{B}$. We refer to $D(A \mid \mathcal{B})$ as the discrete complexity[2] of A with respect to $\mathcal{B}$. It may then be seen the minimum number of AND, OR and NOT gates needed for a Boolean circuit to compute a Boolean function $f : \{0,1\}^m \to \{0,1\}$ is exactly $D(f^{-1}(1) \mid \mathcal{B})$, where $V = \{0,1\}^m$, and $\mathcal{B} = \{x_1, \ldots, x_m\} \subseteq \{0,1\}^m$ contains the input variables $x_1 \ldots, x_m$, seen as subsets of $\{0,1\}^m$ (i.e.. x_i is the set of strings $w \in \{0,1\}^m$ such that $w_i = 1$). So computing the discrete complexity $D(A \mid \mathcal{B})$, for given A and $\mathcal{B}$, generalizes the task of computing the minimum circuit size, by allowing for the consideration of generator sets $\mathcal{B}$ other than $\{x_1, \ldots, x_m\}$.

Definition 3 (Author Definition3)**.** *MDCP is defined as follows:*

Input. *A positive integer n represented in unary describing the size of the ground set $V = [n]$, a target set $A \subseteq V$, a family $\mathcal{B}$ of nonempty subsets of V, and an integer s.*

Output. *The input is accepted if and only if $D(A \mid \mathcal{B}) \leq s$.*

[1] If the parameter s is large then the answer is trivial.

[2] This general setting was already considered in [68], see also Section1.7.2 of Jukna's book [42]. By Stone's representation theorem for Boolean algebras, discrete complexity can be seen as the investigation of circuit complexity with respect to an arbitrary Boolean algebra.

It is easy to see that MDCP is in NP and that it is more general than MCSP. Our hardness result for MDCP, discussed below, also holds under the assumption that the ground set V is a hypercube $\{0,1\}^m$ and that the collection $\mathcal{B}$ contains the sets generated by $x_1, \ldots, x_m$.

2 Preliminaries

We let $[n]$ denote the set $\{1, \ldots, n\}$. We will use the NP-hardness of approximating Set Cover with respect to sets of bounded size [76]. A weaker version of the result from [76] is sufficient. We say that sets $S_1, \ldots, S_\ell$ cover a set T if $T \subseteq S_1 \cup \cdots \cup S_\ell$. For a collection of sets $\mathcal{S}$ and a set T, we use $\text{cover}(T, \mathcal{S})$ to denote the minimum number of sets in $\mathcal{S}$ necessary to cover T.

Definition 7 (Author Definition7: r-Bounded Set Cover Problem)**.** *For a positive integer r, the r-Bounded Set Cover Problem is defined as follows:*

Input. *A positive integer n represented in unary, and a collection $\mathcal{S}$ of nonempty subsets of $[n]$. We are promised that $\bigcup_{S \in \mathcal{S}} S = [n]$ and that $|S| \leq r$ for each $S \in \mathcal{S}$.*

Output. *The value* $\text{cover}([n], \mathcal{S})$.

For convenience, we defined the r-Bounded Set Cover Problem as an optimization problem instead of decision problem.

Theorem 8 (Author Theorem8: Hardness of approximating r-Bounded Set Cover, [76])**.** *For every constant $\alpha \geq 1$ there exists $r \in \mathbb{N}$ such that approximating the r-Bounded Set Cover Problem within a factor of α is NP-hard. More precisely, for every $L \in$ NP, there exists a polynomial-time algorithm that, on input x, outputs a parameter k and an instance $(1^m, \mathcal{S})$ of the r-Bounded Set Cover Problem such that if $x \in L$ then* $\text{cover}([m], \mathcal{S}) \leq k$*, and if $x \notin L$ then* $\text{cover}([m], \mathcal{S}) > \alpha \cdot k$.

3.2 Author warm-up: A reduction from r-Bounded Set Cover to MDCP

We will use instances of MDCP with a particular structure. This makes the second reduction from MDCP to Partial-MCSP more transparent. Let V be a ground set, and $\mathcal{B} = \{B_1, \ldots, B_n\}$ be a collection of nonempty subsets of V. For an element $v \in V$, we use $v^\uparrow \in \{0,1\}^n$ (the "lifted" version of v) to denote the string with the property that $v^\uparrow_i = 1$ if and only if $v \in B_i$. We say that a collection $\mathcal{B}$ is complete (with respect to V) if the following holds: if $a, b \in V$ and $a^\uparrow = b^\uparrow$ then $a = b$. In other words, distinct elements of V have different liftings with respect to $\mathcal{B}$. Our reduction from r-Bounded Set Cover to MDCP will always produce a family of generators that is complete. Let r be a large enough constant, so that say 10-approximating r-Bounded Set Cover is NP-hard. Given an instance $(1^n, \mathcal{S})$ of this problem, the reduction proceed as follows. Fix $V \stackrel{\text{def}}{=} [n^3]$. Partition V into n blocks $V_1, \ldots, V_n$, where $|V_i| = n^2$ for each $i \in [n]$. Given a set $A \subseteq V$, we let $A_i \stackrel{\text{def}}{=} A \cap V_i$. Now view V as the set $\{0,1\}^{3 \log n}$,

and for each $j \in [3 \log n]$, let $B_j = \{v \in V \mid v_j = 1\}$. For convenience, we let $\mathcal{F} = \{B_1, \ldots, B_{3 \log n}\}$. Note that any family $\mathcal{B}$ of generators that contains $\mathcal{F}$ is complete with respect to V. Let $A_{-i} \stackrel{\text{def}}{=} \{A_1, \ldots, A_{i-1}, A_{i+1}, \ldots, A_n\}$. We say that a set A is critical if, for every $i \in [n]$,

$$D(A \mid \mathcal{F} \cup A_{-i}) > n \cdot \log n.$$

It is not hard to show that a uniformly random set $A \subseteq_{1/2} V$ is typically critical.

Lemma 9 (Author Lemma9)**.** *Let $A \subseteq_{1/2} V$ be sampled by letting $v \in A$ independently with probability$1/2$ for each $v \in V$. Then, $\Pr_A[A$ is critical$] \to 1$ as $n \to \infty$.*

Proof. For a fixed $i \in [n]$, we argue below that

$$\Pr_A[D(A \mid \mathcal{F} \cup A_{-i}) \leq n \cdot \log n] = o(1/n).$$

The lemma follows by a union bound. Note that, conditioning on the choice of $A_1, \ldots, A_{i-1}, A_{i+1}, \ldots, A_n$, the set A_i is still a uniformly distributed subset of V_i. Moreover, after we fix $\mathcal{F}$ and A_{-i}, any construction of a set $E \subseteq V$ from $\mathcal{F} \cup A_{-i}$ using s operations can be described by a binary string of length at most $O(s \cdot (\log s + \log n))$. Since $|V_i| = n^2$, the probability that the discrete complexity of A (conditioned on the choice of $A_1, \ldots, A_{i-1}, A_{i+1}, \ldots, A_n$) given $\mathcal{F} \cup A_{-i}$ falls below $s = n \cdot \log n$ is at most

$$\frac{2^{O(s \cdot (\log s + \log n))}}{2^{n^2}} = o(1/n).$$

Since this holds for any choice of $A_1, \ldots, A_{i-1}, A_{i+1}, \ldots, A_n$, the desired probability upper bound holds. □

We assume from now on that we have efficiently produced a critical set A. (Our randomized reduction will always be correct on a critical set.) Note that so far we have only inspected the input 1^n from $(1^n, \mathcal{S})$. Our approximate reduction outputs a triple $(1^{n^3}, A, \mathcal{B}_{\mathcal{S}})$, where A is generated as above, and whose family $\mathcal{B}_{\mathcal{S}}$ is defined as follows. For each set $S \in \mathcal{S}$, let $W_S \stackrel{\text{def}}{=} \bigcup_{i \in S} A_i$. Now set

$$\mathcal{B}_{\mathcal{S}} \stackrel{\text{def}}{=} \mathcal{F} \cup \{W_S \mid S \in \mathcal{S}\}.$$

Clearly, the triple $(1^{n^3}, A, \mathcal{B}_{\mathcal{S}})$ can be efficiently computed from $(1^n, \mathcal{S})$. We argue next that computing $D(A \mid \mathcal{B}_{\mathcal{S}})$ allows us to 2-approximate cover$([n], \mathcal{S})$.

Lemma 10 (Author Lemma10)**.** *If* cover$([n], \mathcal{S}) = \ell$ *then* $D(A \mid \mathcal{B}_{\mathcal{S}}) \leq \ell$.

Proof. Let $S_1, \ldots, S_\ell$ be a cover of $[n]$ using sets from $\mathcal{S}$. Then, by construction of the sets W_S, we get that $A = W_{S_1} \cup \cdots \cup W_{S_\ell}$. In particular, it is possible to construct A using at most ℓ operations (unions) starting from the sets in $\mathcal{B}_{\mathcal{S}}$. □

Lemma 11 (Author Lemma11)**.** *If* $D(A \mid \mathcal{B}_{\mathcal{S}}) = \ell$ *then* cover$([n], \mathcal{S}) \leq 2\ell$.

Proof. Let $t \stackrel{\text{def}}{=} |\mathcal{S}|$, and suppose that $E_1, \ldots, E_\ell$ represent a construction of $A = E_\ell$ from sets in $\mathcal{B}_{\mathcal{S}}$. For convenience, we write $\mathcal{B}_{\mathcal{S}} = \mathcal{F} \cup \{W_{S_1}, \ldots, W_{S_t}\}$. In order to prove Lemma11, we need the following simple claim.

Claim 12 (Author Claim12)**.** *$E_1, \ldots, E_\ell$ depend on at most 2ℓ sets from $\{W_{S_1}, \ldots, W_{S_t}\}$.*

This claim holds simply because each set E_j depends on at most 2 other sets. One may replace 2ℓ by $\ell+1$ by arguing more carefully, that any ℓ-node directed acyclic graph with fan-in 2 and a single sink node has at most $\ell+1$ source nodes. But this tighter bound is unnecessary for our purpose. Continuing with the proof of Lemma11, and relabelling some indexes if necessary, we assume for convenience of notation that the construction of A from $\mathcal{B}_{\mathcal{S}}$ depends only on $\mathcal{F} \cup \{W_{S_1}, \ldots, W_{S_{2\ell}}\}$. In other words,

$$D(A \mid \mathcal{F} \cup \{W_{S_1}, \ldots, W_{S_{2\ell}}\}) \leq \ell.$$

Since $W_{S_j} = \bigcup_{i \in S_j} A_i$ and $|S_j| \leq r$, we get that $D(W_{S_j} \mid \{A_i\}_{i \in S_j}) \leq r$. In turn, by composing constructions, we obtain that

$$D(A \mid \mathcal{F} \cup \{A_i\}_{i \in S_1} \cup \cdots \cup \{A_i\}_{i \in S_{2\ell}}) \leq \ell \cdot r.$$

We can assume that $r < \log n$ for a large enough n. In addition, notice that if $\ell > n$ then the statement of Lemma11 is trivial, since by assumption $\mathcal{S}$ always covers $[n]$ and as a consequence $\text{cover}([n], \mathcal{S}) \leq n$. But from $r < \log n$ and $\ell \leq n$ it follows that the expression above can be upper bounded by $n \cdot \log n$. Given that A is critical, it must be the case that the sets $S_1, \ldots, S_{2\ell}$ cover $[n]$. In other words, $\text{cover}([n], \mathcal{S}) \leq 2\ell$, which completes the proof. □

Consequently, it immediately follows from Lemmas9,10,and11 and from Theorem8 that MDCP is NP-hard to compute under polynomial-time randomized reductions.

4 Main result: NP-hardness of circuit minimization for multi-output functions

4.1 Definitions

4.1.1 Multi-output Functions, Concatenations, and Truth Tables

For a multi-output Boolean function $f : \{0,1\}^n \to \{0,1\}^m$, the components of f are the single-output functions $f_1, \ldots, f_m$ where $f_i : \{0,1\}^n \to \{0,1\}$ is defined so $f_i(x)$ equals the ith output bit of $f(x)$. For a multi-output Boolean function f, we let the circuit complexity of f, denoted $\text{CC}(f)$, be the minimum size of any circuit computing f. For strings $x, y \in \{0,1\}^*$, we let $x \bullet y$ denote the concatenated string. We identify a multi-output Boolean function $f : \{0,1\}^n \to \{0,1\}^m$ with the concatenated string $T_1 \bullet \cdots \bullet T_m \in \{0,1\}^{m \cdot 2^n}$ where $T_1, \ldots, T_m$ are the truth tables of the components $f_1, \ldots, f_m$ respectively. If $f : \{0,1\}^n \to \{0,1\}^{m_1}$ and $g : \{0,1\}^n \to \{0,1\}^{m_2}$ are Boolean functions with the same number of inputs, we define the concatenated Boolean function $f \bullet g : \{0,1\}^n \to \{0,1\}^{m_1+m_2}$ given by $f \bullet g(x) = f(x) \bullet g(x)$. We also use the $\bullet$ symbol to indicate concatenation in a similar way that $\sum$ acts for addition. For example, if $T_1, \ldots, T_m$ are truth tables of functions with the same number of inputs, then we use the notation $\bullet_{i \in [m]} T_i$ to indicate $T_1 \bullet \cdots \bullet T_m$.

4.1.2 The Evaluation Function and Multi-output Computation

Each circuit C induces a multi-output Boolean function we call the Evaluation Function of C, denoted Eval-C, that computes the outputs of each of the gates in C. In more detail, for a circuit C that takes n inputs and has s gates, the evaluation function induced by C, denoted Eval-$C : \{0,1\}^n \to \{0,1\}^{s+n}$, is given by $x_1 \bullet \cdots \bullet x_n \bullet g_1 \bullet \cdots \bullet g_s$ where x_i is the function computed by the ith input wire of C and g_j is the function computed by the jth gate in C (for this to be well-defined, we need to fix an ordering of the gates of C, but, for our purposes, any ordering will do). Using the Evaluation Function, an equivalent definition of multi-output circuit computation to the one given in the introduction is that a Boolean circuit C computes a (multi-output) Boolean function f if and only if every component of f is a component of Eval-C.

4.1.3 Windows of Truth Tables

Given a truth table T of length n and a subset $S \subseteq [n]$, we define the S-window of T, denoted $T\langle S\rangle$, to be the truth table of length n that (informally) "sees" T on the elements of S and zeroes everywhere else. Rigorously,

$$T\langle S\rangle(x) = \begin{cases} T(x) & \text{if } x \in S, \\ 0 & \text{otherwise.} \end{cases}$$

4.1.4 Canonical DNF Circuits

For each (single output) Boolean function $f : \{0,1\}^n \to \{0,1\}$, it will be useful to fix an algorithm that outputs a single "canonical" Boolean circuit for computing f. For our purposes, many algorithms are possible, but, for concreteness, we will define the canonical circuit of a Boolean function $f : \{0,1\}^n \to \{0,1\}$ to be the naive DNF formula for f, denoted DNF_f, given by

$$\mathrm{DNF}_f \stackrel{\text{def}}{=} ((x_1 = y_1^1) \wedge \cdots \wedge (x_n = y_n^1)) \vee \cdots \vee ((x_1 = y_1^t) \wedge \cdots \wedge (x_n = y_n^t))$$

where

- $y^1, \ldots, y^t$ are the YES inputs of f in lexicographical order,
- $x_1, \ldots, x_n$ index the bits of the input string x,
- $y_1^j, \ldots, y_n^j$ index the bits of y^j for each $j \in [t]$,
- and for $i \in [n]$ and $j \in [t]$, $(x_i = y_i^j)$ is syntax for $\begin{cases} x_i & \text{if } y_i^j = 1, \\ \neg x_i & \text{if } y_i^j = 0. \end{cases}$

It is easy to see that DNF_f can be computed in polynomial-time given the truth table of f. Moreover, reading the above definition of DNF_f from left to right gives a natural way of defining the kth gate in DNF_f, whereby the kth gate corresponds to the kth gate symbol appearing in the above formula. This fact will later be useful in our analysis.

4.1.5 Lifting Sets

Our reduction will use a way to lift subsets into subsets on larger ground sets. To do this, we will first define a canonical partition of $[m]$ into n parts for $m \geq n$. Let $\mathcal{P}^{m,n} = (P_1^{m,n}, \ldots, P_n^{m,n})$ be the partition of $[m]$ into n parts given by

$$P_i^{m,n} = \{j \in [m] : j \equiv i \pmod{n}\}.$$

From this partition, we can now lift subsets as follows. Let $n \leq m \in \mathbb{N}$. Let $S \subseteq [n]$. The m-lift of S, denoted S^m, is the set given by

$$S^m \stackrel{\text{def}}{=} \bigcup_{i \in S} P_i^{m,n}.$$

4.2 A reduction from r-Bounded Set Cover to Multi-MCSP

In order to show that Multi-MCSP is NP-hard, we give a probabalistic polynomial-time many-one reduction with one-sided error from a constant approximation of r-Bounded Set Cover to Multi-MCSP. In fact, for convenience, our reduction will be from the optimization version of approximating r-Bounded Set Cover to the optimization version of Multi-MCSP (computing CC), but it will be easy to see that our reduction can be converted into the desired reduction for the corresponding decision problems. Let r be a large enough constant, so that say 10-approximating r-Bounded Set Cover is NP-hard. Given an instance $(1^n, \mathcal{S})$ of this problem, the reduction proceeds as follows. Let $m = O(n^3)$ be the least power of two greater than n^3. Let T be a uniformly random truth table of length m. I.e., T is a binary string in $\{0,1\}^m$, representing a function from $\{0,1\}^{\log m}$ to $\{0,1\}$. Compute the truth table of

$$g \stackrel{\text{def}}{=} \bullet_{S \in \mathcal{S}} \text{Eval-DNF}_{T\langle S^m \rangle}.$$

Let k be the number of distinct components of g that are not functions computed by an input gate, that is,

$$k \stackrel{\text{def}}{=} |\{g_i : g_i \text{ is a component of } g \text{ and } g_i \neq x_j \text{ for all } j \in [\log m]\}|.$$

The reduction then outputs[3]

$$\Delta \stackrel{\text{def}}{=} \text{CC}(T \bullet g) - k.$$

First, we argue that this procedure runs in polynomial time. The only two steps that may raise concern are whether we can compute the truth table of g efficiently and whether we can compute k efficiently. To show that the truth table of g can be computed efficiently, it suffices to show that for each $S \in \mathcal{S}$, the truth table of $\text{Eval-DNF}_{T\langle S^m \rangle}$ can be computed in time polynomial in n. Computing $T\langle S^m \rangle$ can be done in time $O(m + |T|) = O(m)$ and outputs a truth table of length m. The canonical DNF of the truth table $T\langle S^m \rangle$ can be computed in time polynomial in $|T\langle S^m \rangle| = m$, and the resulting DNF has $\log m$ inputs and size at most $O(m \log m)$. Finally, computing the Evaluation

[3]For the decision problems, we can determine if there is a 10-approximate set cover of size ℓ by outputting $\text{Multi-MCSP}(T \bullet g, k + \ell)$, which computes whether $\Delta \leq \ell$.

Function of a circuit with $\log m$ inputs and $O(m \log m)$ gates can be done in time $O(m^3)$ by just evaluating the circuit on every input. Hence, putting these all together, computing the truth table of Eval - $\mathrm{DNF}_{T\langle S^m \rangle}$ can be done in time polynomial in $m = O(n^3)$. To see that we can compute k efficiently, realize that we have already computed the full truth table of g and that removing any components computed by one of the $\log m$ input wires along with any duplicate components takes time at most quadratic in the length of the truth table of g. Now, we will argue for the correctness of the reduction. We will show that

$$\underbrace{\mathrm{cover}([n], \mathcal{S})/4 - 4}_{\substack{\text{with high probability} \\ \text{using Lemma16}}} \leq \Delta \leq \underbrace{\mathrm{cover}([n], \mathcal{S})}_{\substack{\text{unconditionally} \\ \text{using Lemma15}}},$$

and thus, with high probability Δ computes a 10-approximation of r-Bounded Set Cover when n is sufficiently large.[4] Moreover, this computation has one-sided error since the upper bound holds unconditionally. Thus, after proving Lemmas 15 and 16, we will have shown that there is a randomized polynomial-time many-one reduction with one-sided error from 10-approximating r-Bounded Set Cover to Multi-MCSP. Before proving Lemmas 15 and 16, we make the following observation about computing g.

Proposition 14 (Author Proposition14)**.** *If a circuit C computes g, then there are k distinct gates in C that compute components of g. Moreover,* $\mathrm{CC}(g) = k$.

Proof. We begin by proving the first statement, which also implies the lower bound $\mathrm{CC}(g) \geq k$. Suppose C is a circuit that computes g. Then every distinct component of g has a (necessarily distinct) input wire or gate from C that computes that component. Therefore, since g has k distinct components that are not computed by an input wire, C must have at least k distinct gates computing components of g. Next, we sketch the proof of the upper bound $\mathrm{CC}(g) \leq k$. Let C be the circuit built as follows. For each $S \in \mathcal{S}$, iterate through the gates g in $\mathrm{DNF}_{T\langle S^m \rangle}$ in topological order. Let $\diamond \in \{\wedge, \vee, \neg\}$ be the gate type of g. If g computes a function that is already computed by C, then ignore it. Otherwise, add a $\diamond$ gate to C that takes as input(s) those gate(s) in C that compute the function(s) which are fed as inputs to g in $\mathrm{DNF}_{T\langle S^m \rangle}$ (we are guaranteed to find such gates in C since we are iterating in topological order). By construction, C computes g. (Recall that $g = \bullet_{S \in \mathcal{S}}$ Eval - $\mathrm{DNF}_{T\langle S^m \rangle}$, and our construction ensures C computes every function computed by a gate in $\mathrm{DNF}_{T\langle S^m \rangle}$ for any $S \in \mathcal{S}$.) Moreover, our construction maintains that every gate in C computes a component of g that is not computed by any other gate or input wire. Thus, since g has at most k unique components not computed by input wires, C has at most k gates. □

One consequence of Proposition14 is that $\Delta = \mathrm{CC}(T \bullet g) - \mathrm{CC}(g)$. With this fact, we can prove our two main lemmas.

Lemma 15 (Author Lemma15)**.** *If* $\mathrm{cover}([n], \mathcal{S}) = \ell$*, then* $\Delta \leq \ell$.

[4]Since $\mathrm{cover}([n], \mathcal{S}) \geq n/r$ (using that the sets in $\mathcal{S}$ have cardinality at most r), we will actually get that, for each $\epsilon > 0$, Δ gives a $4 + \epsilon$ approximation with high probability when n is sufficiently large.

Proof. Let $S_1, \ldots, S_\ell$ be a cover of $[n]$ using sets from $\mathcal{S}$. Then, by construction, we have that $T = T\langle S_1^m \rangle \vee \cdots \vee T\langle S_\ell^m \rangle$. Since $T\langle S_1^m \rangle, \ldots, T\langle S_\ell^m \rangle$ are components of g, this implies that

$$\Delta = \mathrm{CC}(T \bullet g) - \mathrm{CC}(g) \leq \ell$$

as desired. □

Lemma 16 (Author Lemma16)**.** *Let ℓ be the largest integer such that* $\mathrm{cover}([n], \mathcal{S}) \geq 4\ell$*. Then, $\Delta > \ell$ with high probability.*

Proof. Our strategy will be as follows. We say that the choice of T is bad if, for that choice of T, $\Delta \leq \ell$. We will then upper bound the number of bad T by showing such T have short descriptions. Fix some bad T. Then $\ell \geq \Delta = \mathrm{CC}(T \bullet g) - k$, so $\mathrm{CC}(T \bullet g) \leq \ell + k$. Since there is a circuit C computing $T \bullet g$ using at most $\ell + k$ gates and k of the gates in C must compute the unique components of g (using Proposition14), it follows that there is a circuit D that takes $(\log(m) + k)$-inputs and has at most ℓ gates such that

$$D(x, g_1(x), \ldots, g_k(x)) = T(x)$$

for all $x \in \{0,1\}^{\log m}$ where $g_1, \ldots, g_k$ are the unique components of g. Moreover, since D has only ℓ gates of fan-in 2, it uses at most 2ℓ of the components of g in the circuit. Thus, after a possible relabeling of $g_1, \ldots, g_k$, we can assume D takes at most $(\log(m) + 2\ell)$-inputs and that

$$D(x, g_1(x), \ldots, g_{2\ell}(x)) = T(x).$$

Hence, to describe T, we just need to have a description for D as well as a description for $g_1, \ldots, g_{2\ell}$. Indeed, this will be the last step in our eventual description of T. We present the eventual description now so as to guide the reader. Our proof will subsequently proceed working through this description from bottom to top.

1. Given
   - a subset $J \subseteq [n]$ of size at most $\leq n(1 - \frac{1}{2r})$
   - for each $j \in J$ a partial truth table encoding $(j, V_j) \in [n] \times \{0,1\}^{m/n+1}$
   - r-bounded subsets $S_1, \ldots, S_{2\ell} \subseteq [n]$ whose union is J,
   - gate numbers $u_1, \ldots, u_{2\ell} \in [2m \log m]$,
   - and a circuit D of size ℓ with $(n + 2\ell)$ inputs
2. For $j \in J$, let $T\langle P_j^{m,n} \rangle$ be the function whose values on $P_j^{m,n}$ in lexicographic order are given by the binary string V_j and is zero everywhere else
3. For $i \in [2\ell]$, let $T\langle S_i^m \rangle = \bigvee_{j \in S_i \subseteq J} T\langle P_j^{m,n} \rangle$
4. For $i \in [2\ell]$, let g_i be the function computing by the u_ith gate of $\mathrm{DNF}_{T\langle S_i^m \rangle}$
5. Let $T(x) = D(x, g_1(x), \ldots, g_{2\ell}(x))$

Step4: Describing the g_i Since $g_1, \ldots, g_{2\ell}$ are components of g and $g = \bullet_{S \in \mathcal{S}} \text{Eval-DNF}_{T\langle S^m \rangle}$, there exist $u_1, \ldots, u_{2\ell}$ and $S_1, \ldots, S_{2\ell}$ such that each g_i is the u_ith gate of $\text{DNF}_{T\langle S_i^m \rangle}$ for $i \in [2\ell]$. Moreover, each $u_i \leq 2m \log m$ by the trivial upper bound on the number of gates in a canonical DNF.

Step3: Describing the $T\langle S_i^m \rangle$ Next, we focus on encoding $T\langle S_i^m \rangle$ for some i. Since $S_i^m = \bigcup_{j \in S_i} P_j^{m,n}$ (by construction of S_i^m), we have (by construction of $T\langle S_i^m \rangle$) that $T\langle S_i^m \rangle = \bigvee_{j \in S_i} T\langle P_j^{m,n} \rangle$. Thus, to compute $T\langle S_i^m \rangle$ for all $i \in [2\ell]$, it suffices to know $S_1, \ldots, S_{2\ell}$ as well as $T\langle P_j^{m,n} \rangle$ for all $j \in J \stackrel{\text{def}}{=} \bigcup_{i \in [2\ell]} S_i$.

Step2: Describing $T\langle P_j^{m,n} \rangle$ for $j \in J$ The key to our encoding is to realize that $|J|$ cannot be too large, and thus, we do not need to know $T\langle P_j^{m,n} \rangle$ for all $j \in [n]$. Since $\text{cover}([n], \mathcal{S}) \geq 4\ell$, and J is the union of 2ℓ sets from $\mathcal{S}$, it follows that $|J| \leq n - 2\ell$. Moreover, we have that

$$n/r \leq \text{cover}([n], \mathcal{S}) < 4\ell + 4$$

where the first inequality comes from the sets in $\mathcal{S}$ having cardinality at most r and the second inequality comes from the definition of ℓ. Thus,

$$|J| \leq n - 2\ell < n - \frac{n}{2r} + 2 = n(1 - \frac{1}{2r}) + 2.$$

Moreover, we can encode $T\langle P_j^{m,n} \rangle$ for $j \in J$ very efficiently. To describe $T\langle P_j^{m,n} \rangle$, it suffices to describe j and then give the list of values of $T\langle P_j^{m,n} \rangle$ on the set $P_j^{m,n}$ in lexicographic order. Since $|P_j^{m,n}| \leq m/n + 1$ (essentially by construction), we can encode this list of values by a string $V_j \in \{0,1\}^{m/n+1}$ (where we pad this string with extra zeroes if $|P_j^{m,n}| < m/n + 1$).

Step1: Counting the bits in the description Now, we count the number of bits in our description of T. Describing J requires n-bits. For $j \in J$, each partial truth table encoding (j, V_j) requires at most $2 \log n + m/n + 1$ bits. Using that $|J| \leq n(1 - \frac{1}{2r})$, we get that all these encodings require at most $n(1 - \frac{1}{2r})(2 \log n + m/n + 1)$ bits. Encoding the r-bounded subsets $S_1, \ldots, S_{2\ell} \subseteq [n]$ requires at most $2n\ell \leq n^2$ bits (here we use the fact that $4\ell \leq n$ since $\text{cover}([n], \mathcal{S}) = 4\ell$). Encoding the gate numbers $u_1, \ldots, u_{2\ell}$ requires at most $4\ell \log(2m \log m) = O(n \log n)$ (using that $4\ell \leq n$ and that $m = n^{O(1)}$). Finally, describing a circuit with ℓ gates and $(n + 2\ell)$ input bits where $4\ell \leq n$ requires $O(n \log n)$ bits. Putting this all together and using that $m = O(n^3)$, we get that T can be described using

$$\begin{aligned} &n + (n(1 - \tfrac{1}{2r}) + 2)(2 \log n + m/n + 1) + n^2 + O(n \log n) \\ &\quad = (1 - \tfrac{1}{2r})m + O(n^2) = (1 - \Omega(1))m \end{aligned}$$

bits. Thus, the number of bad T is upper bounded by $2^{(1-\Omega(1))m}$, so the probability that T is bad is at most $2^{-\Omega(m)}$. □

References

[1] Misha Alekhnovich, Mark Braverman, Vitaly Feldman, Adam R. Klivans, and Toniann Pitassi. The complexity of properly learning simple concept classes. Journal of Computer and System Sciences, 74(1):16–34, 2008.

[2] Eric Allender and Bireswar Das. Zero knowledge and circuit minimization. In International Symposium on Mathematical Foundations of Computer Science (MFCS), pages 25–32, 2014.

[3] Eric Allender, Joshua A. Grochow, Dieter van Melkebeek, Cristopher Moore, and Andrew Morgan. Minimum circuit size, graph isomorphism, and related problems. SIAM Journal on Computing, 47(4):1339–1372, 2018.

[4] Eric Allender, Lisa Hellerstein, Paul McCabe, Toniann Pitassi, and Michael E. Saks. Minimizing disjunctive normal form formulas and AC0 circuits given a truth table. SIAM Journal on Computing, 38(1):63–84, 2008.

[5] Eric Allender and Shuichi Hirahara. New insights on the (non-)hardness of circuit minimization and related problems. In International Symposium on Mathematical Foundations of Computer Science (MFCS), pages 54:1–54:14, 2017.

[6] Eric Allender, Dhiraj Holden, and Valentine Kabanets. The minimum oracle circuit size problem. Computational Complexity, 26(2):469–496, 2017.

[7] Eric Allender, Rahul Ilango, and Neekon Vafa. The non-hardness of approximating circuit size. In International Computer Science Symposium in Russia (CSR), pages 13–24, 2019.

[8] Anurag Anshu, Naresh Goud Boddu, and Dave Touchette. Quantum log-approximate-rank conjecture is also false. arXiv, 2018. arXiv:1811.10525.

[9] Benny Applebaum, Boaz Barak, and David Xiao. On basing lower-bounds for learning on worst-case assumptions. In Symposium on Foundations of Computer Science (FOCS), pages 211–220, 2008.

[10] Theodore P. Baker, John Gill, and Robert Solovay. Relativizations of the P =? NP question. SIAM Journal on Computing, 4(4):431–442, 1975.

[11] Avrim Blum, Merrick L. Furst, Michael J., and Richard J. Lipton. Cryptographic primitives based on hard learning problems. In International Cryptology Conference (CRYPTO), pages 278–291, 1993.

[12] Avrim L. Blum and Ronald L. Rivest. Training a 3-node neural network is np-complete. Neural Networks, 5(1):117–127, 1992.

[13] Anselm Blumer, Andrzej Ehrenfeucht, David Haussler, and Manfred K. Warmuth. Occam's razor. Information Processing Letters, 24(6):377–380, 1987.

[14] Andrej Bogdanov and Luca Trevisan. Average-case complexity. Foundations and Trends in Theoretical Computer Science, 2(1), 2006.

[15] Dan Boneh and Richard J. Lipton. Amplification of weak learning under the uniform distribu- tion. In Conference on Learning Theory (COLT), pages 347–351, 1993.

[16] Joan Boyar, Philip Matthews, and René Peralta. On the shortest linear straight-line program for computing linear forms. In International Symposium on Mathematical Foundations of Computer Science (MFCS), pages 168–179, 2008.

[17] Marco L. Carmosino, Russell Impagliazzo, Valentine Kabanets, and Antonina Kolokolova. Learning algorithms from natural proofs. In Conference on Computational Complexity (CCC), pages 10:1–10:24, 2016.

[18] Amit Chakrabarti and Oded Regev. An optimal lower bound on the communication complexity of gap-Hamming-distance. SIAM Journal on Computing, 41(5):1299–1317, 2012. doi:10. 1137/120861072.

[19] Arkadev Chattopadhyay, Nikhil S Mande, and Suhail Sherif. The log-approximate-rank conjecture is false. In Symposium on Theory of Computing (STOC), pages 42–53. ACM, 2019.

[20] Mahdi Cheraghchi, Valentine Kabanets, Zhenjian Lu, and Dimitrios Myrisiotis. Circuit lower bounds for MCSP from local pseudorandom generators. Electronic Colloquium on Computational Complexity (ECCC), 26:22, 2019.

[21] Gil Cohen and Igor Shinkar. The complexity of DNF of parities. In Innovations in Theoretical Computer Science (ITCS), pages 47–58, 2016.

[22] Sebastian Czort. The complexity of minimizing disjunctive normal form formulas. Master's Thesis, University of Aarhus, 1999.

[23] Uriel Feige. A threshold of ln n for approximating set cover. Journal of the ACM, 45(4):634–652, 1998.

[24] Uriel Feige and Joe Kilian. Zero knowledge and the chromatic number. Journal of Computer and System Sciences, 57(2):187–199, 1998.

[25] Magnus Gausdal Find, Alexander Golovnev, Edward A. Hirsch, and Alexander S. Kulikov. A better-than-3n lower bound for the circuit complexity of an explicit function. In Symposium on Foundations of Computer Science (FOCS), pages 89–98, 2016.

[26] Michael R. Garey and David S. Johnson. Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman, 1979.

[27] Alexander Golovnev, Rahul Ilango, Russell Impagliazzo, Valentine Kabanets, Antonina Koloko- lova, and Avishay Tal. AC0 [p] lower bounds against MCSP via the coin problem. Electronic Colloquium on Computational Complexity (ECCC), 26:18, 2019.

[28] Mika Göös and Toniann Pitassi. Communication lower bounds via critical block sensitivity. SIAM Journal on Computing, 47(5):1778–1806, 2018. doi:10.1137/16M1082007.

[29] Gary D. Hachtel and Fabio Somenzi. Logic synthesis and verification algorithms. Springer, 2006.

[30] Thomas R. Hancock, Tao Jiang, Ming Li, and John Tromp. Lower bounds on learning decision lists and trees. Information and Computation, 126(2):114–122, 1996.

[31] Johan Hastad. Clique is hard to approximate within $n^{1-\varepsilon}$. In Symposium on Foundations of Computer Science (FOCS), pages 627–636, 1996.

[32] Shuichi Hirahara. Non-black-box worst-case to average-case reductions within NP. In Sym- posium on Foundations of Computer Science (FOCS), pages 247–258, 2018.

[33] Shuichi Hirahara, Igor Carboni Oliveira, and Rahul Santhanam. NP-hardness of minimum circuit size problem for OR-AND-MOD circuits. In Computational Complexity Conference (CCC), pages 5:1–5:31, 2018.

[34] Shuichi Hirahara and Rahul Santhanam. On the average-case complexity of MCSP and its variants. In Computational Complexity Conference (CCC), pages 7:1–7:20, 2017.

[35] Shuichi Hirahara and Osamu Watanabe. Limits of minimum circuit size problem as oracle. In Conference on Computational Complexity (CCC), pages 18:1–18:20, 2016.

[36] John M. Hitchcock and Aduri Pavan. On the NP-completeness of the minimum circuit size problem. In Conference on Foundation of Software Technology and Theoretical Computer Science (FSTTCS), pages 236–245, 2015.

[37] Sangxia Huang. Improved hardness of approximating chromatic number. In International Con- ference on Approximation Algorithms for Combinatorial Optimization Problems (APPROX), pages 233–243, 2013.

[38] Rahul Ilango. AC0 [p] lower bounds and NP-hardness for variants of MCSP. Electronic Colloquium on Computational Complexity (ECCC), 26:21, 2019.

[39] Russell Impagliazzo, Valentine Kabanets, and Ilya Volkovich. The Power of Natural Properties as Oracles. In Computational Complexity Conference (CCC), volume 102, pages 7:1–7:20, 2018.

[40] Kazuo Iwama and Hiroki Morizumi. An explicit lower bound of 5n − o(n) for boolean circuits. In International Symposium on Mathematical Foundations of Computer Science (MFCS), pages 353–364, 2002.

[41] J. Stephen Judd. Learning in networks is hard. In International Conference on Neural Networks (ICNN), volume 2, pages 685–692, 1987.

[42] Stasys Jukna. Boolean function complexity: advances and frontiers, volume 27. Springer, 2012.

[43] Valentine Kabanets and Jin-yi Cai. Circuit minimization problem. In Symposium on Theory of Computing (STOC), pages 73–79, 2000.

[44] Maurice Karnaugh. The map method for synthesis of combinational logic circuits. Transactions of the American Institute of Electrical Engineers, Part I: Communication and Electronics, 72(5):593–599, 1953.

[45] Michael J. Kearns and Leslie G. Valiant. Cryptographic limitations on learning boolean formulae and finite automata. J. ACM, 41(1):67–95, 1994. doi:10.1145/174644.174647.

[46] Michael J. Kearns and Umesh V. Vazirani. An Introduction to Computational Learning Theory. MIT Press, 1994.

[47] Subhash Khot. Improved inapproximability results for maxclique, chromatic number and approximate graph coloring. In Symposium on Foundations of Computer Science (FOCS), pages 600–609, 2001.

[48] Subhash Khot and Rishi Saket. Hardness of minimizing and learning DNF expressions. In Symposium on Foundations of Computer Science (FOCS), pages 231–240, 2008.

[49] Ker-I Ko. On the complexity of learning minimum time-bounded Turing machines. SIAM Journal on Computing, 20(5):962–986, 1990.

[50] Pravesh K. Kothari and Roi Livni. Improper learning by refuting. In Innovations in Theoretical Computer Science (ITCS), pages 55:1–55:10, 2018.

[51] Jan Krajícek. Forcing with Random Variables and Proof Complexity. Cambridge University Press, 2011.

[52] Eyal Kushilevitz. Communication complexity. In Advances in Computers, volume 44, pages 331–360. Elsevier, 1997.

[53] Eyal Kushilevitz and Noam Nisan. Communication Complexity. Cambridge University Press, 1997.

[54] Eyal Kushilevitz and Enav Weinreb. On the complexity of communication complexity. In Symposium on Theory of Computing (STOC), pages 465–474, 2009.

[55] Leonid Levin. Universal sequential search problems. Problemy Peredachi Informatsii, 9(3):115– 116, 1973.

[56] László Lovász and Michael Saks. Lattices, mobius functions and communications complexity. In Symposium on Foundations of Computer Science (FOCS), pages 81–90, 1988.

[57] Carsten Lund and Mihalis Yannakakis. On the hardness of approximating minimization problems. Journal of the ACM, 41(5):960–981, 1994.

[58] William J. Masek. Some NP-complete set covering problems. Unpublished Manuscript, 1979.

[59] Edward J. McCluskey. Introduction to the theory of switching circuits. McGraw-Hill, 1965.

[60] Dylan M. McKay, Cody D. Murray, and R. Ryan Williams. Weak lower bounds on resource- bounded compression imply strong separations of complexity classes. In Symposium on Theory of Computing (STOC), 2019.

[61] Moritz Müller and Ján Pich. Feasibly constructive proofs of succinct weak circuit lower bounds. Electronic Colloquium on Computational Complexity (ECCC), 24:144, 2017.

[62] Cody D. Murray and Richard Ryan Williams. On the (non) NP-hardness of computing circuit complexity. In Conference on Computational Complexity (CCC), pages 365–380, 2015.

[63] Igor Carboni Oliveira, Ján Pich, and Rahul Santhanam. Hardness magnification near state-of- the-art lower bounds. In Conference on Computational Complexity (CCC), 2019.

[64] Igor Carboni Oliveira and Rahul Santhanam. Conspiracies between learning algorithms, circuit lower bounds, and pseudorandomness. In Computational Complexity Conference (CCC), pages 18:1–18:49, 2017.

[65] Igor Carboni Oliveira and Rahul Santhanam. Hardness magnification for natural problems. In Symposium on Foundations of Computer Science (FOCS), pages 65–76, 2018.

[66] James Orlin. Contentment in graph theory: covering graphs with cliques. Indagationes Mathematicae, 80(5):406–424, 1977.

[67] Leonard Pitt and Leslie G. Valiant. Computational limitations on learning from examples. Journal of the ACM, 35(4):965–984, 1988.

[68] Pavel Pudlák, Vojtech Rödl, and Petr Savický. Graph complexity. Acta Informatica, 25(5):515– 535, 1988.

[69] Netanel Raviv. Truth table minimization of computational models. CoRR/arXiv, abs/1306.3766, 2013. arXiv:1306.3766.

[70] Alexander A. Razborov and Steven Rudich. Natural proofs. Journal of Computer and System Sciences, 55(1):24–35, 1997.

[71] Ronald L. Rivest. Learning decision lists. Machine Learning, 2(3):229–246, 1987.

[72] J. Paul Roth and Richard M. Karp. Minimization over boolean graphs. IBM Journal of Research and Development, 6(2):227–238, 1962.

[73] Christoph Scholl. Functional decomposition with applications to FPGA synthesis. Kluwer Academic Publishers, 2001.

[74] Makrand Sinha and Ronald de Wolf. Exponential separation between quantum communication and logarithm of approximate rank. arXiv, 2018. arXiv:1811.10090.

[75] Boris A Trakhtenbrot. A survey of Russian approaches to perebor (brute-force search) algorithms. Annals of the History of Computing, 6(4):384–400, 1984.

[76] Luca Trevisan. Non-approximability results for optimization problems on bounded degree instances. In Symposium on Theory of Computing (STOC), pages 453–461, 2001.

[77] Christopher Umans, Tiziano Villa, and Alberto L. Sangiovanni-Vincentelli. Complexity of two-level logic minimization. IEEE Trans. on CAD of Integrated Circuits and Systems, 25(7):1230–1246, 2006.

[78] Salil P. Vadhan. On learning vs. refutation. In Conference on Learning Theory (COLT), pages 1835–1848, 2017.

[79] Marcin Wrochna and Stanislav Živnỳ. Improved hardness for H-colourings of G-colourable graphs. arXiv, 2019. arXiv:1907.00872.

[80] Andrew Chi-Chih Yao. Some complexity questions related to distributive computing (pre- liminary report). In Symposium on Theory of Computing (STOC), pages 209–213, 1979. doi:10.1145/800135.804414.

[81] Andrew Chi-Chih Yao. Theory and applications of trapdoor functions (extended abstract). In Symposium on Foundations of Computer Science (FOCS), pages 80–91, 1982.

[82] David Zuckerman. Linear degree extractors and the inapproximability of max clique and chromatic number. In Symposium on Theory of Computing (STOC), pages 681–690, 2006.